# src/evaluation.py

Does the agent beat naive RAG — and by how much?

This module is the methodology that carries the project. It measures retrieval and answers separately: retrieval metrics run fully locally, and answer quality is graded by an LLM judge (key-gated and cached), comparing naive RAG against the agent, sliced by single-hop vs multi-hop.

Walkthrough of all 197 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Evaluation — does the agent actually beat naive RAG, and by how much?
 2
 3  This is the methodology track that carries the project. It has two halves:
 4
 5  1. **Retrieval metrics (fully local, no API key).** For each evaluation question
 6     we know which paper(s) genuinely answer it. We measure whether retrieval
 7     surfaced them:
 8       - hit@k   : did *any* relevant paper appear in the top-k?
 9       - recall@k: what fraction of the relevant papers appeared?
10       - MRR     : 1 / rank of the first relevant paper (rewards ranking it high).
11     These use only the vector store, so they reproduce exactly on any machine.
12
13  2. **Answer quality (LLM-as-judge, key-gated + cached).** A separate Claude call
14     grades each answer for *groundedness* (is every claim supported by the sources?)
15     and *correctness* (does it actually answer the question?), for naive RAG and
16     the agent, split by single-hop vs multi-hop. The grades are cached and
17     committed, so the headline comparison reproduces from cache without a key.
18
19  The design goal is a fair head-to-head: same corpus, same retriever, same
20  questions — the only thing that differs is single-shot vs iterative retrieval.
21  """
```

Lines 1 to 21 are the module docstring. They lay out the two halves — local retrieval metrics (hit@k, recall@k, MRR against known-relevant papers) and LLM-as-judge answer grading (groundedness and correctness) — and state the design goal: a fair head-to-head where the only thing that differs between the two systems is single-shot versus iterative retrieval.

```
22
23  from __future__ import annotations
24
25  import json
26  from dataclasses import dataclass
27
28  from . import agent as agent_mod
29  from . import config, llm, rag
30  from .vectorstore import VectorStore
```

Lines 22 to 30 are imports: json for reading the question set and parsing judge output (line 25), dataclass (line 26), the agent module aliased as agent_mod (line 28), config plus the llm wrapper and rag module (line 29), and VectorStore (line 30).

```
31
32
33  # ---------------------------------------------------------------------------- #
34  # Evaluation set
35  # ---------------------------------------------------------------------------- #
36  @dataclass
37  class EvalQuestion:
38      """One graded question.
39
40      `relevant_ids` are the arXiv ids that genuinely answer it — the ground truth
41      for retrieval metrics. `hop` tags the question as needing one fact
42      ("single") or several ("multi"); the whole hypothesis is that the agent's
43      edge shows up on the multi-hop rows.
44      """
45
46      qid: str
47      question: str
48      relevant_ids: list[str]
49      hop: str  # "single" or "multi"
50
51
```

Lines 31 to 51 define the EvalQuestion dataclass — one graded question. The docstring (lines 38 to 44) explains relevant_ids is the ground truth (the arXiv ids that genuinely answer it) and hop tags it single or multi, with the hypothesis that the agent's edge appears on multi-hop rows. Fields are qid, question, relevant_ids, and hop (lines 46 to 49).

```
52  def load_eval_set() -> list[EvalQuestion]:
53      """Load the curated question set from eval/qa_set.json."""
54      with open(config.EVAL_DIR / "qa_set.json", encoding="utf-8") as f:
55          raw = json.load(f)
56      return [EvalQuestion(**q) for q in raw]
```

Lines 52 to 56 define load_eval_set, which reads eval/qa_set.json (line 54), JSON-loads it (line 55), and constructs one EvalQuestion per entry by unpacking each dict (line 56).

```
57
58
59  # ---------------------------------------------------------------------------- #
60  # Retrieval metrics (local)
61  # ---------------------------------------------------------------------------- #
62  def hit_at_k(retrieved_ids: list[str], relevant_ids: list[str]) -> float:
63      """1.0 if any relevant id is in the retrieved set, else 0.0."""
64      return 1.0 if set(retrieved_ids) & set(relevant_ids) else 0.0
65
```

Lines 57 to 65 define hit_at_k. Line 64 returns 1.0 if the intersection of retrieved and relevant ids is non-empty, else 0.0 — a blunt did-retrieval-find-anything measure. Lines 59 to 61 are the section header for the local retrieval metrics.

```
66
67  def recall_at_k(retrieved_ids: list[str], relevant_ids: list[str]) -> float:
68      """Fraction of the relevant ids that were retrieved."""
69      if not relevant_ids:
70          return 0.0
71      found = set(retrieved_ids) & set(relevant_ids)
72      return len(found) / len(set(relevant_ids))
73
```

Lines 66 to 73 define recall_at_k. Line 69 to 70 guard against an empty relevant set (returning 0.0). Line 71 computes the intersection; line 72 returns its size over the number of distinct relevant ids — the fraction of needed papers that were retrieved, the metric that exposes multi-hop weakness.

```
74
75  def mrr(retrieved_ids: list[str], relevant_ids: list[str]) -> float:
76      """Reciprocal rank of the first relevant id (0 if none retrieved).
77
78      Ranks are 1-based, so a relevant paper in the top slot scores 1.0, second
79      slot 0.5, and so on — rewarding retrieval for putting the right paper high,
80      not merely somewhere in the list.
81      """
82      relevant = set(relevant_ids)
83      for rank, aid in enumerate(retrieved_ids, start=1):
84          if aid in relevant:
85              return 1.0 / rank
86      return 0.0
87
88
```

Lines 74 to 88 define mrr (mean reciprocal rank). The docstring (lines 76 to 81) explains ranks are 1-based so the top slot scores 1.0, the second 0.5, and so on. Lines 82 to 85 walk the retrieved ids in order and return one over the rank of the first relevant one; line 86 returns 0.0 if none is relevant. It rewards ranking the right paper high.

```
89  def retrieval_ids(store: VectorStore, question: str, k: int = config.TOP_K) -> list[str]:
90      """The ordered list of distinct arXiv ids retrieved for a question.
91
92      Chunks are de-duplicated to paper level (keeping first appearance) because
93      the ground truth is per-paper: two chunks from the same paper are one hit.
94      """
95      ids: list[str] = []
96      for h in store.search(question, k=k):
97          if h.arxiv_id not in ids:
98              ids.append(h.arxiv_id)
99      return ids
```

Lines 89 to 99 define retrieval_ids, which turns a search into an ordered list of distinct paper ids. The docstring (lines 90 to 94) explains the de-duplication to paper level: the ground truth is per-paper, so two chunks from one paper count as one hit. Lines 95 to 99 iterate the search hits and append each arxiv_id only the first time it appears, preserving rank order.

```
100
101
102 def evaluate_retrieval(store: VectorStore, questions: list[EvalQuestion],
103                        k: int = config.TOP_K) -> list[dict]:
104     """Compute hit@k, recall@k, and MRR for every question. Fully local."""
105     rows = []
106     for q in questions:
107         got = retrieval_ids(store, q.question, k=k)
108         rows.append(
109             {
110                 "qid": q.qid,
111                 "hop": q.hop,
112                 "hit@k": hit_at_k(got, q.relevant_ids),
113                 "recall@k": recall_at_k(got, q.relevant_ids),
114                 "mrr": mrr(got, q.relevant_ids),
115             }
116         )
117     return rows
```

Lines 100 to 117 define evaluate_retrieval, which computes all three metrics for every question — fully local, no key. For each question (line 106) it gets the retrieved paper ids (line 107) and appends a row with the qid, hop, and the three metric values (lines 108 to 116). Line 117 returns the rows, ready to aggregate or plot.

```
118
119
120 # ---------------------------------------------------------------------------- #
121 # Answer quality (LLM-as-judge)
122 # ---------------------------------------------------------------------------- #
123 JUDGE_SYSTEM = (
124     "You are a strict evaluator of answers to research questions. You are given a "
125     "question, a set of source passages, and an answer. Grade two things on a 1-5 "
126     "scale:\n"
127     "  groundedness: 5 = every claim is directly supported by the passages; "
128     "1 = the answer makes claims the passages do not support.\n"
129     "  correctness: 5 = fully and accurately answers the question; "
130     "1 = wrong or off-topic.\n"
131     "Reply with ONLY a JSON object: "
132     '{\"groundedness\": <int>, \"correctness\": <int>}.'
133 )
```

Lines 118 to 133 define JUDGE_SYSTEM, the system prompt for the LLM judge. It instructs the model to grade two axes on a 1-to-5 scale — groundedness (are all claims supported by the passages) and correctness (does it accurately answer) — and to reply with ONLY a JSON object of the two integer scores, which keeps the output machine-parseable.

```
134
135
136 def judge_answer(question: str, sources_text: str, answer_text: str) -> dict:
137     """Grade one answer with the LLM judge; returns groundedness + correctness.
138
139     Cached and key-gated like every other LLM call, so the graded results
140     reproduce from the committed cache without an API key.
141     """
142     user = (
143         f"Question: {question}\n\n"
144         f"Source passages:\n{sources_text}\n\n"
145         f"Answer:\n{answer_text}\n\n"
146         "Grade the answer. Reply with only the JSON object."
147     )
148     result = llm.call(
149         messages=[{"role": "user", "content": user}],
150         system=JUDGE_SYSTEM,
151         model=config.JUDGE_MODEL,
152     )
153     text = llm.first_text(result)
154     try:
155         grades = json.loads(text)
156         return {
157             "groundedness": int(grades["groundedness"]),
158             "correctness": int(grades["correctness"]),
159         }
160     except (json.JSONDecodeError, KeyError, ValueError):
161         # A judge that returns unparseable output scores 0 — visible and honest,
162         # rather than silently dropping the row.
163         return {"groundedness": 0, "correctness": 0}
```

Lines 134 to 163 define judge_answer, which grades one answer. Lines 142 to 147 build the user message containing the question, the source passages, and the answer. Lines 148 to 152 call the LLM with the judge system prompt and judge model (cached and key-gated like every call). Line 153 pulls the text; lines 154 to 159 parse the JSON and return the two integer scores. Lines 160 to 163 catch a malformed judge reply and score it 0 — visible and honest, rather than silently dropping the row.

```
164
165
166  def evaluate_answers(store: VectorStore, questions: list[EvalQuestion]) -> list[dict]:
167      """Run naive RAG and the agent on every question and grade both answers.
168
169      Produces one row per (question, system) so the notebook can compare naive vs
170      agent, sliced by hop type. Also records the agent's step count — the token /
171      latency cost of its extra retrievals, which is the other side of the trade.
172      """
173      rows = []
174      for q in questions:
175          naive = rag.answer(store, q.question)
176          agentic = agent_mod.answer(store, q.question)
177
178          naive_sources = rag.format_context(naive.sources)
179          agent_sources = rag.format_context(agentic.sources)
180
181          rows.append(
182              {
183                  "qid": q.qid, "hop": q.hop, "system": "naive",
184                  "n_retrievals": 1,
185                  "n_cited": len(naive.cited_ids),
186                  **judge_answer(q.question, naive_sources, naive.answer),
187              }
188          )
189          rows.append(
190              {
191                  "qid": q.qid, "hop": q.hop, "system": "agent",
192                  "n_retrievals": len(agentic.steps),
193                  "n_cited": len(agentic.cited_ids),
194                  **judge_answer(q.question, agent_sources, agentic.answer),
195              }
196          )
197      return rows
```

Lines 164 to 197 define evaluate_answers, the head-to-head harness. For each question (line 174) it runs naive RAG (line 175) and the agent (line 176) on the same store, formats each system's sources (lines 178 to 179), and appends two rows — one per system. The naive row records one retrieval (lines 181 to 188); the agent row records its actual step count as n_retrievals (lines 189 to 195), which is the cost side of the trade. Each row merges in the judge's scores via dict unpacking. Line 197 returns all rows, giving one row per question-and-system for slicing by hop type in the notebook.